

From a Scalar CFG Language to Datalog

Xin Zhang

Contents

1	Overview	2
2	Scalar CFG	2
2.1	Syntax	2
2.2	Semantics	3
3	Intermediate Language	3
3.1	Syntax	3
3.2	Semantics	3
4	Datalog	4
4.1	Syntax	4
4.2	Semantics	5
5	CFG-to-IR Compiler	5
5.1	Completeness	5
5.2	Soundness	6
6	IR-to-Datalog Compiler	6
6.1	Completeness	6
6.2	Soundness	6
7	Connecting the Phases	6
7.1	Completeness	6
7.2	Soundness	7

1 Overview

This document is a top-level description of a compiler from a language with control-flow-graph constructs to Datalog. The ultimate goal is to compile from an imperative language with collection types (e.g., sets of records) to Datalog. As the resulting Datalog program needs to track mutable program states, we associate the states with timestamps. Control-flow graphs help set clear boundaries where the timestamps should be incremented. In an initial effort to develop a formally verified compiler from a CFG language with collection types to Datalog, I found it difficult to prove the soundness of the compiler. It was necessary to show the independence between subsets of rules in the Datalog program in order to invoke the inductive hypothesis of the proof, but that was hard because timestamp manipulation and relation name generation were intertwined with other aspects of the compiler definition. Therefore, I decided to decouple the concerns using an intermediate language. To experiment with the idea, I simplified the source language to handle only scalar data and developed a two-phase compiler, with formal proofs of its completeness and soundness. While such a scalar imperative language can be compiled to a simpler Datalog program where a single relation encodes the entire state (with one argument per mutable variable), we represent each mutable variable with a relation, which will become necessary when we eventually equip the CFG language with collection types again.

2 Scalar CFG

A program in the source language consists of a static component and a dynamic one. The static component has a program signature $\vec{t}$ representing the types of the mutable variables and a list of blocks giving a control-flow graph. A block simultaneously assigns to all mutable variables and performs a control flow operation. A control flow operation can unconditionally jump to a block, branching to one of two blocks based on the value of an expression, or returning from the program. This language is scalar in that all the expressions compute only scalar values rather than, e.g., sets of values. The dynamic component has a list of values $\vec{v}$ representing the values of the mutable variables at the beginning of program execution, and a pointer p indicating the next block to be executed. Note that the dynamic component can also be viewed as the state of the machine at some point of program execution, so the semantics of a program can be written as a trace of such states, with $p = \text{none}$ in a terminating state.

2.1 Syntax

Expression	$\mathcal{E}$	$::=$	$x \mid b \mid n \mid \neg \mathcal{E} \mid \mathcal{E} \wedge \mathcal{E} \mid \mathcal{E} + \mathcal{E} \mid \mathcal{E} < \mathcal{E} \mid \mathcal{E} = \mathcal{E}$
Flow	$\mathcal{F}$	$::=$	$\text{goto } n \mid \text{if } \mathcal{E} \text{ then } n \text{ else } n \mid \text{ret}$
Block	$\mathcal{B}$	$::=$	$\langle \text{asgns} : \vec{\mathcal{E}}, \text{flow} : \mathcal{F} \rangle$
Value	$\mathcal{V}$	$::=$	$n \mid b$
Type	$\mathcal{T}$	$::=$	$\text{int} \mid \text{bool}$
Pointer	p	$::=$	$\text{some } n \mid \text{none}$
CFG	$\mathcal{G}_s$	$::=$	$\langle \text{sig} : \vec{\mathcal{T}}, \text{blks} : \vec{\mathcal{B}} \rangle$
State	$\mathcal{G}_d$	$::=$	$\langle \text{str} : \vec{\mathcal{V}}, \text{ptr} : p \rangle$
Program	$\mathcal{P}$	$::=$	$\langle \text{static} : \mathcal{G}_s, \text{dynamic} : \mathcal{G}_d \rangle$

The well-typedness of a program requires that all assignments and the program state respect the program signature, and that branch condition expressions have Boolean types. Moreover, all block indexes in a well-typed program must be less than the total number of blocks, i.e., there is no out-of-range block index.

2.2 Semantics

The interpretation of an expression given a value environment for mutable variables $\llbracket e \rrbracket_{\vec{v}}$ is what one would expect. Similarly, it is straightforward to define the interpretation of a control flow operation F given a program state G_d , written as $\llbracket F \rrbracket_{G_d}$, as a pointer value. Using these two definitions, one can easily define how a block B transforms a program state G_d to give a new program state G'_d , written as $G'_d = \llbracket B \rrbracket_{G_d}$, with the interpretation returning **none** if $G_d.\text{ptr} = \text{none}$. One execution step of a program $\langle \text{static} : G_s, \text{dynamic} : G_d \rangle$ can then be defined as follows.

$$\llbracket G_s \rrbracket_{G_d} = \text{match } G_d.\text{ptr} \text{ with some } n \rightarrow \llbracket G_s.\text{blks}[n] \rrbracket_{G_d} \mid \text{none} \rightarrow \text{none end}$$

Iterating the computation T times, we obtain the result of executing a program for T steps.

$$\begin{aligned} \llbracket G_s \rrbracket_{G_d}^0 &= \text{some } G_d \\ \llbracket G_s \rrbracket_{G_d}^{T+1} &= \text{match } \llbracket G_s \rrbracket_{G_d}^T \text{ with some } G'_d \rightarrow \llbracket G_s \rrbracket_{G'_d} \mid \text{none} \rightarrow \text{none end} \end{aligned}$$

For brevity, we write $\llbracket P \rrbracket^T$ for $\llbracket G_s \rrbracket_{G_d}^T$ where $P = \langle \text{static} : G_s, \text{dynamic} : G_d \rangle$.

3 Intermediate Language

The syntax of a program in the intermediate language resembles that of the source language, with expressions replaced by lists of Datalog rules. We call every such list a module. Different modules are interpreted independently. This separation enables us to reason about expression compilation independently from the control-flow machinery, which simplifies the soundness proof.

We refer to the intermediate language as IR in the rest of this document.

3.1 Syntax

Global relation	R_{glob}	$::=$	$R_{\text{mut}}^n \mid R_{\text{ter}} \mid R_{\text{blk}}^n$
Relation	R	$::=$	$R_{\text{glob}} \mid R_{\text{aux}}^n$
Function	h	$::=$	$\text{lit}^b \mid \text{lit}^z \mid \text{lit}^n \mid \text{not} \mid \text{and} \mid \text{plus} \mid \text{lt} \mid \text{eq}$
Variable	X	$::=$	x_n
Expression	E	$::=$	$X \mid \langle h, \vec{E} \rangle$
Clause	C	$::=$	$\langle R, \vec{E} \rangle$
Rule	$\mathcal{R}$	$::=$	$\langle \text{concl} : C, \text{hyps} : \vec{C} \rangle$
Module	$\mathcal{M}$	$::=$	$\vec{\mathcal{R}}$
Control flow	$\mathcal{F}$	$::=$	$\text{goto } n \mid \text{if } \mathcal{M} \text{ then } n \text{ else } n \mid \text{ret}$
Block	$\mathcal{B}$	$::=$	$\langle \text{asgns} : \vec{\mathcal{R}}, \text{flow} : \mathcal{F} \rangle$
Program	$\mathcal{P}$	$::=$	$\langle \text{init} : \mathcal{M}, \text{blks} : \vec{\mathcal{B}} \rangle$

3.2 Semantics

The semantics of the IR is closer to that of Datalog, as the interpretation of a program is a database. We have values and facts defined as follows.

$$\begin{aligned} \text{Value } V &::= b \mid z \mid n \\ \text{Fact } F &::= \langle R, \vec{V} \rangle \end{aligned}$$

A database is represented as a set of facts. Suppose we have an input database ϕ . The interpretation of a module M , written as $\llbracket M \rrbracket(\phi)$, is simply the proof-theoretic semantics of the module, i.e., the set of facts derivable by the rules in the module given the extensional database

ϕ . The “output” of the module is stored in the R_{aux}^0 relation used to define the semantics of blocks.

For a list of modules denoting simultaneous assignments to the mutable variables, the interpretation is about the new values of the variables. Suppose there are N mutable variables, then

$$\llbracket \vec{M} \rrbracket(\phi) = \{ \langle R_{\text{mut}}^x, \vec{v} \rangle \mid \langle R_{\text{aux}}^0, \vec{v} \rangle \in \llbracket \vec{M}[x] \rrbracket(\phi), x < N \}$$

where $\vec{M}[x]$ denotes the x -th element of $\vec{M}$.

The interpretation of a control flow operation F is about the next active block.

$$\llbracket F \rrbracket(\phi) = \begin{cases} \{ \langle R_{\text{blk}}^k, [] \rangle \} & F = \text{goto } k \\ \{ \langle R_{\text{blk}}^{k_1}, [] \rangle \mid \langle R_{\text{aux}}^0, [\text{true}] \rangle \in \llbracket M \rrbracket(\phi) \} \cup \\ \quad \{ \langle R_{\text{blk}}^{k_2}, [] \rangle \mid \langle R_{\text{aux}}^0, [\text{false}] \rangle \in \llbracket M \rrbracket(\phi) \} & F = \text{if } M \text{ then } k_1 \text{ else } k_2 \\ \{ \langle R_{\text{ter}}^k, [] \rangle \} & F = \text{ret} \end{cases}$$

Given a list of blocks $\vec{B}$ that comprise the control-flow graph, we can compute the database resulting from T steps of program execution.

$$\begin{aligned} \llbracket \vec{B} \rrbracket^0(\phi) &= \phi \\ \llbracket \vec{B} \rrbracket^{T+1}(\phi) &= \{ \llbracket \vec{M} \rrbracket(\psi) \cup \llbracket F \rrbracket(\psi) \mid \psi = \llbracket \vec{B} \rrbracket(\phi), \langle R_{\text{blk}}^k, [] \rangle \in \psi, \vec{B}[k] = \langle \vec{M}, F \rangle \} \end{aligned}$$

The interpretation of a program $P = \langle \text{init} : M_0, \text{blks} : \vec{B} \rangle$ is the interpretation of its blocks starting from the database derived from the initial module.

$$\llbracket P \rrbracket^T = \llbracket \vec{B} \rrbracket^T(\llbracket M_0 \rrbracket(\{\}))$$

4 Datalog

The Datalog language shares the same global relations as the IR. The types of values that an expression can take are also identical to those in the IR. That makes it easier to define equivalence between programs in the two languages.

The auxiliary relations are now annotated with a block ID, a usage tag that corresponds to either a mutable variable or a branching condition, and a local ID that is inherited from the IR. The annotations allow us to distinguish which blocks and assignments produced which auxiliary facts. The compiler from the IR to Datalog will rename the auxiliary relations such that relations in separate modules remain conditionally independent, where the “conditions” refer to the global relations.

The variables that appear in the rules remain untouched, but the Datalog language contains a special timestamp variable that is used exclusively in the first argument of relations.

Unlike the IR, a program in the Datalog language is simply a list of rules without any additional structure.

4.1 Syntax

Global relation	R_{glob}	$::=$	$R_{\text{mut}}^n \mid R_{\text{ter}} \mid R_{\text{blk}}^n$
Auxiliary relation usage	u	$::=$	$\text{asgn } n \mid \text{cond}$
Auxiliary relation tag	A	$::=$	$\langle \text{blk} : n, \text{use} : u, \text{id} : n \rangle$
Relation	R	$::=$	$R_{\text{glob}} \mid R_{\text{aux}}^A$
Function	h	$::=$	$\text{lit}^b \mid \text{lit}^z \mid \text{lit}^n \mid \text{not} \mid \text{and} \mid \text{plus} \mid \text{lt} \mid \text{eq}$
Variable	X	$::=$	$x_n \mid x_{\text{time}}$
Expression	E	$::=$	$X \mid \langle h, \vec{E} \rangle$
Clause	C	$::=$	$\langle R, \vec{E} \rangle$
Rule	$\mathcal{R}$	$::=$	$\langle \text{concl} : C, \text{hyps} : \vec{C} \rangle$
Program	$\mathcal{P}$	$::=$	$\vec{\mathcal{R}}$

4.2 Semantics

The semantics of a Datalog program P is the typical proof-theoretic semantics starting with an empty extensional database, which we write as $\llbracket P \rrbracket(\{\})$.

5 CFG-to-IR Compiler

This phase of the compiler focuses on the compilation of expressions to Datalog subprograms we call modules. Expressions assigned to distinct variables or in distinct blocks are compiled to relations in separate modules, and modules do not interact with one another. The target program retains the shape of blocks from the source program while transitioning to Datalog-like semantics. The interpretation of the IR depends on the proof-theoretic semantics of the modules, which are Datalog programs. The semantics of the IR is defined as databases at different timestamps.

The compiler is said to be correct if every well-typed source program is compiled to a target program such that their interpretations satisfy some notion of equivalence. Due to the interpretation of an IR program as a set of facts, we state the correctness property as two components in opposite directions. The completeness direction says that the interpretation of the source program can be “lowered” to a set of global facts, all of which must be in the interpretation of the target program. The soundness direction says that any global facts in the interpretation of the target program can be obtained by interpreting the source program and lowering that interpretation.

5.1 Completeness

Suppose there are N mutable variables in the source program. A value environment $\vec{v}$ is lowered to the set of facts

$$(\vec{v}) \downarrow = \{ \langle R_{\text{mut}}^x, l' \rangle \mid x < N, l' = [(\vec{v}[x]) \downarrow] \}$$

where $(v) \downarrow$ denotes the lowering of a value in the source language to one in the target language. For a source language with records and sets, the lowering may produce a set of lists of values instead.

A pointer p is lowered as follows.

$$(p) \downarrow = \text{match } p \text{ with some } n \rightarrow \{ \langle R_{\text{blk}}^n, [] \rangle \} \mid \text{none} \rightarrow \{ \langle R_{\text{ter}}, [] \rangle \} \text{ end}$$

As such, a program state G_d is lowered to a set of facts defined by

$$(G_d) \downarrow = (G_d.\text{str}) \downarrow \cup (G_d.\text{ptr}) \downarrow$$

The compiler completeness theorem states that for any well-typed source program $P = \langle \text{static} : G_s, \text{dynamic} : G_d \rangle$, it is compiled to a program P' such that for every timestamp T ,

$$\llbracket P \rrbracket^T = \text{some } (G'_d) \implies (G'_d) \downarrow \subseteq \llbracket P' \rrbracket^T$$

For symmetry with the soundness theorem below, let us define the lowering of an optional state q by

$$(q) \downarrow = \text{match } q \text{ with some } G_d \rightarrow (G_d) \downarrow \mid \text{none} \rightarrow \{ \}$$

The completeness theorem can then be rewritten to

$$(\llbracket P \rrbracket^T) \downarrow \subseteq \llbracket P' \rrbracket^T$$

5.2 Soundness

Reusing the definitions from above, the compiler soundness theorem states that for any well-typed source program $P = \langle \text{static} : G_s, \text{dynamic} : G_d \rangle$, it is compiled to a program P' such that for every timestamp T ,

$$\llbracket P' \rrbracket^T \subseteq (\llbracket P \rrbracket^T) \downarrow$$

6 IR-to-Datalog Compiler

This phase of the compiler implements the assignments and control flow operations in the semantics of the IR via Datalog rules. Relations are renamed so all modules and the additional rules co-exist in a flat Datalog program without ambiguity. Since the semantics no longer have the built-in notion of timestamps, timestamps are now handled explicitly as the first argument of all relations.

Compiler correctness now associates the database obtained by executing the source program for T steps with the semantics of the target program restricted to the global variables and the timestamp T .

Some well-formedness properties of the source program are required for the correctness property to be provable.

- The initialization module must consist of rules where the conclusions have global relation names and the lists of hypotheses are empty.
- The assignment module must consist of rules where the conclusions have auxiliary relation names and singleton argument lists.

6.1 Completeness

The completeness theorem states that every well-formed source program P' is compiled to a target program $P'' = (P') \downarrow$ such that

$$\forall T, R, l. \text{glob}(R) \implies \langle R, l \rangle \in \llbracket P' \rrbracket^T \implies \langle R, T :: l \rangle \in \llbracket P'' \rrbracket(\{\})$$

where $\text{glob}(R)$ checks that R is a global relation name.

6.2 Soundness

The soundness theorem states that every well-formed source program P' is compiled to a target program P'' such that

$$\forall R, l'. \text{glob}(R) \implies \langle R, l' \rangle \in \llbracket P'' \rrbracket(\{\}) \implies \exists T, l. l' = T :: l \wedge \langle R, l \rangle \in \llbracket P' \rrbracket^T$$

7 Connecting the Phases

The two phases are composed to give the compiler from the CFG language to Datalog. The well-formedness properties of the IR program required by the correctness proofs of the second phase are guaranteed by the first phase of the compiler.

7.1 Completeness

The top-level completeness theorem states that for every well-typed program P in the CFG language, lowering it through the two phases gives a Datalog program $P'' = ((P) \downarrow) \downarrow$ such that

$$\forall T, R, l. \text{glob}(R) \implies \langle R, l \rangle \in (\llbracket P \rrbracket^T) \downarrow \implies \langle R, T :: l \rangle \in \llbracket P'' \rrbracket(\{\})$$

7.2 Soundness

The top-level soundness theorem states that every well-typed program P in the CFG language, lowering it through the two phases gives a Datalog program $P'' = ((P) \downarrow) \downarrow$ such that

$$\forall R, l'. \text{glob}(R) \implies \langle R, l' \rangle \in \llbracket P'' \rrbracket(\{\}) \implies \exists T, l. l' = T :: l \wedge \langle R, l \rangle \in (\llbracket P \rrbracket^T) \downarrow$$